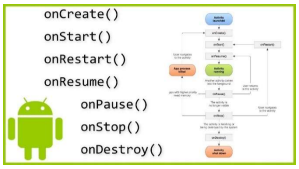
93

Chapitre 3

Activités Android

Naoufel KHAYATI
naoufel.khayati@eniso.u-sousse.tn



AU. 2025 - 2026

Introduction

94

- Une activité peut être assimilée à un écran qu'une application propose à son utilisateur.
- Pour chaque écran de l'application, on doit donc créer une activité.
- La transition entre deux écrans correspond au lancement d'une activité (via des intents) ou au retour sur une activité placée en arrière-plan.

Introduction

95

- Une activité à base d'XML Views est composée de deux volets:
  - Sa logique métier et la gestion de son cycle de vie qui sont implémentées dans une classe héritant de Activity (ou de l'une de ses sous-classes).
  - Son interface utilisateur définie dans un fichier XML placé dans les ressources de l'application (dans res/layout).

PLAN

96

- **Cycle de vie des activités**
  - Pile d'Activités
  - Cycle de vie
  - Sauvegarde de l'état d'une activité
- **UI des activités**
  - Views
    - Manipulation des objets d'interfaces
    - Gestion des évènements
  - Layouts
    - Types de layouts
    - Configuration du ConstraintLayout

97

Cycle de vie des activités

(Valable aussi pour les activités Jetpack Compose)

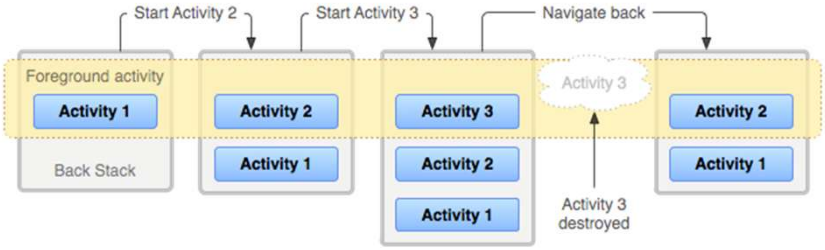
Pile d'activités

98

- Si un utilisateur reçoit un appel, il devient plus important qu'il puisse y répondre que d'écouter la musique ou de naviguer sur le web.
- Les développeurs d'Android ont pris deux décisions :
  - À tout moment l'application peut laisser sa place à d'autres priorités.
    - Si l'application (en background) utilise trop de ressources empêchant le système de fonctionner correctement, Android pourra décider de l'arrêter sans prévenir.
  - L'activité connaitra plusieurs états au cours de son cycle de vie (active, en pause, etc.).
- Les activités d'une application sont planifiées en utilisant la notion de pile.
  - Exp : Quand une activité se lance, elle se met tout en haut d'une pile dite « **pile d'activités** ».

Pile d'activités

99

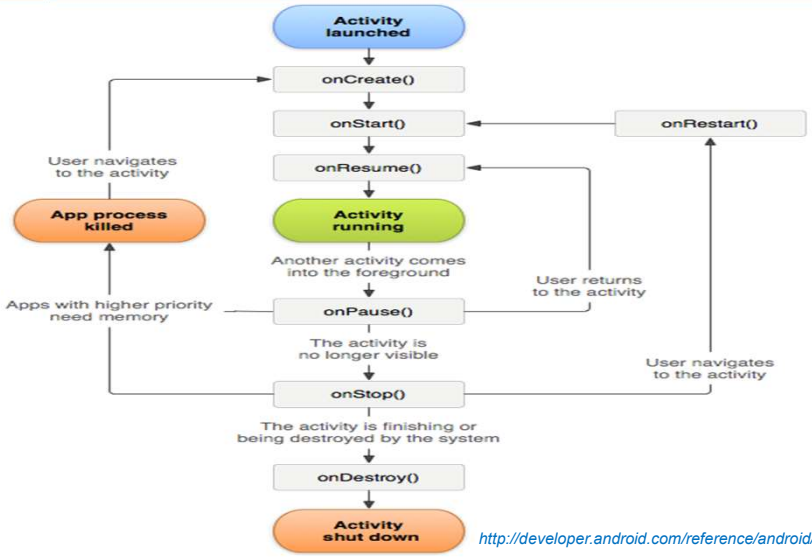


- Chaque nouvelle activité d'une application s'ajoute comme un élément à la pile.
- Lorsque l'utilisateur appuie sur le bouton « Retour », l'activité en cours est détruite et l'activité précédente reprend.

http://developer.android.com/guide/components/tasks-and-back-stack.html

Cycle de vie

100



http://developer.android.com/reference/android/app/Activity.html

Cycle de vie

101

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
    override fun onStart() {
        super.onStart()
    }
    override fun onResume() {
        super.onResume()
    }
    override fun onPause() {
        super.onPause()
    }
    override fun onStop() {
        super.onStop()
    }
    override fun onDestroy() {
        super.onDestroy()
    }
    override fun onRestart() {
        super.onRestart()
    }
}
```

Cycle de vie

102

onCreate()

- Exécutée quand l'utilisateur clique sur l'icône de l'application.
- Sert à créer l'activité en initialisant l'interface graphique.
- Reçoit toujours en paramètre un objet Bundle qui contient l'état dans lequel était l'activité avant l'invocation.

onStart()

- Appelée avant que l'activité ne soit visible (début du passage au premier plan).
  - Si l'activité apparaît en premier plan, c'est **onResume()** qui est invoquée.
  - Si l'activité ne peut pas aller en avant plan quelle que soit la raison, l'activité sera transférée à **onStop()**.

Cycle de vie

103

onResume()

- Appelée au moment où l'application passe en avant plan.
- À la fin de l'appel à la méthode **onResume()**, l'application se trouve au premier plan et reçoit les interactions utilisateurs.

onPause()

- Appelée lorsque l'activité passe en arrière plan (une autre activité prend sa place en avant plan).
- Une fois cette méthode exécutée, Android peut tuer à tout moment l'activité sans prévenir.
- La nouvelle activité ne démarrera pas tant que l'exécution de cette fonction n'est pas terminée.

Cycle de vie

104

onStop()

- Appelée quand l'activité n'est plus du tout visible quelle que soit la raison.
- Dans cette méthode, on doit arrêter tous les traitements et services exécutés par l'application.

onRestart()

- Exécutée lorsque l'activité arrêtée redémarre (repasse en premier plan).

onDestroy()

- Appelée quand l'application sera totalement fermée (Processus terminé).
  - Effectue un nettoyage final avant qu'une activité ne soit détruite (arrêt de threads, par exemple).
  - Cela peut se produire soit parce que l'activité se termine (appel de **finish()** dessus), ou parce que le système a décidé de détruire cette instance de l'activité pour économiser de l'espace.
- Toutes les données non sauvegardées seront perdues.

Sauvegarde de l'état d'une activité

105

- **Cas de l'appui sur les touches « Back » / « Home »**
  - *« Back » pour revenir vers une activité ou application précédente : Android détruit l'activité qui était en cours.*
    - Si on revient vers l'activité quittée, cette dernière est recréée de nouveau, donc rien n'est restauré.
  - *« Home » pour revenir sur le bureau : Android peut décharger le contenu des éléments d'interface de l'activité quittée afin de* gagner des ressources en mémoire.
    - Si on revient vers l'activité quittée, cette dernière peut avoir perdu les valeurs saisies dans les zones de texte, par exemple.

Sauvegarde de l'état d'une activité

106

- **Cas de rotation de l'écran / changement de langue**
  - Android adapte l'interface dans ces deux cas en arrêtant l'activité puis en la redémarrant.
  - Ce redémarrage peut provoquer la perte des changements apportés à l'activité en cours puisqu'il s'agit d'une recréation de l'interface graphique.

Sauvegarde de l'état d'une activité

107

- Par défaut, le système conserve l'état d'une activité quittée.
  - Lorsque l'utilisateur navigue vers une activité précédente, son interface utilisateur s'affiche de la façon dont il l'a laissée.
- Ceci est possible grâce à l'objet **Bundle** passé en paramètre à la méthode onCreate().
  - Cet objet permet de restaurer les valeurs d'interface d'une activité qui a été déchargée de la mémoire.
  - **Attention !!!** Un tel objet ne peut sauvegarder que le contenu des objets graphiques ayant un identifiant (ID).

108

UI des activités

IHM sous Android

109

- Le nom du fichier xml *permet de retrouver le layout dans le code Kotlin à travers* ***R.layout.XML_file_name.***

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

  - Associe la vue graphique à l'activité.

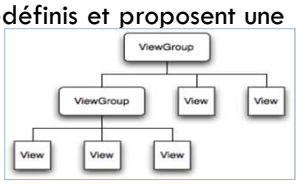
IHM sous Android

110

- Les éléments graphiques sont définis dans le package **android.widget.**
  - Regroupe l'affichage ainsi que la gestion des évènements liés à la zone d'affichage.

```
import android.widget.Button
import android.widget.TextView
import android.widget.RelativeLayout
```

- Une ou plusieurs vues peuvent être regroupées dans un « **ViewGroup** » (Layout / Gabarit).
  - Des groupes de vues particuliers sont prédéfinis et proposent une prédisposition des objets graphiques.



IHM sous Android

111

- ViewGroup: les Layouts.
  - LinearLayout
  - RelativeLayout
  - FrameLayout
  - TableLayout
  - GridLayout
  - ConstraintLayout
- View: les Widgets.
  - TextView (Label)
  - EditText
  - Button
  - CheckBox
  - RadioButton
  - ImageButton: Une image qui se comporte comme un bouton.
  - TimePicker / DatePicker: Sélection de l'heure / de date.
  - Etc.

Views

112

- TextView

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/entrer_nom"
    android:textSize="@dimen/taille"
    android:textColor="@color/rouge"
    android:layout_alignParentStart="true"
    android:layout_marginStart="41dp"
    android:layout_marginTop="43dp"
    android:id="@+id/textView" />
```

Entrer un Nom:

colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="colorPrimary">#3F51B5</color>
    <color name="colorPrimaryDark">#303F9F</color>
    <color name="colorAccent">#FF4081</color>
    <color name="rouge">#FF0000</color>
</resources>
```

strings.xml

```
<resources>
    <string name="app_name">Chap4_exp</string>
    <string name="entrer_nom">Entrer un Nom:</string>
    <string name="nom">Nom</string>
    <string name="valider">Valider</string>
</resources>
```

dimens.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <dimen name="taille">15sp</dimen>
</resources>
```

Views

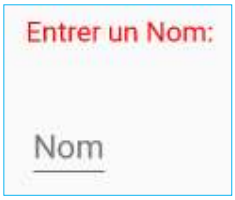
113

- EditText

```
<EditText
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:hint="@string/nom"
    android:layout_marginTop="32dp"
    android:id="@+id/editText"
    android:layout_below="@+id/textView"
    android:layout_alignStart="@+id/textView" />
```

strings.xml

```
<resources>
    <string name="app_name">Chap4_exp</string>
    <string name="entrer_nom">Entrer un Nom:</string>
    <string name="nom">Nom</string>
    <string name="valider">Valider</string>
</resources>
```



Views

114

- Button

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="23dp"
    android:text="@string/valider"
    android:layout_below="@+id/editText"
    android:layout_alignStart="@+id/editText" />
```

strings.xml

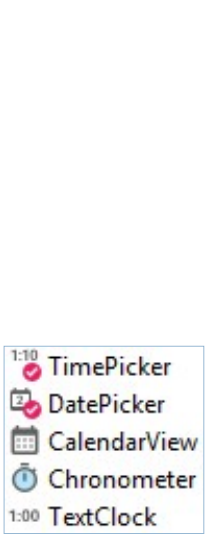
```
<resources>
    <string name="app_name">Chap4_exp</string>
    <string name="entrer_nom">Entrer un Nom:</string>
    <string name="nom">Nom</string>
    <string name="valider">Valider</string>
</resources>
```



Views

115

- CheckBox
- RadioButton
- RadioGroup
- ImageView
- ImageButton
- VideoView
- Widgets pour l'heure et la date



Manipulation des Views – findViewById()

116

- Les composants graphiques présents dans les layouts sont identifiés par un id
  - Ils sont créés dans le fichier xml de layout comme suit : android:id="@+id/textView1"

```
<TextView
    android:id="@+id/textView1"
/>
```

  - Ils étaient principalement accessibles depuis les sources Kotlin et Java via la méthode **findViewById()**
    - KOTLIN : val tvK = findViewById<TextView>(R.id.textView1)
    - JAVA : TextView tvJ = (TextView) findViewById (R.id.textView1);
  - Cette méthode est toujours valable, mais elle est couteuse en lignes de code.

Manipulation des Views – Kotlin extensions

117

- Google a ajouté les KAE (Kotlin Android Extensions) pour s'en passer de findViewById() et utiliser directement l'id donné dans le fichier XML comme identifiant de la vue dans le fichier Kotlin de l'activité.
  - Ça ne marche que pour KOTLIN.
- Il suffit d'avoir le plugin suivant au niveau du fichier build.gradle (niveau app).

```
'kotlin-android-extensions'
```

- Puis d'utiliser directement textView1 comme nom de l'objet Kotlin.
  - Kotlin importera automatiquement le package nécessaire

```
import kotlinx.android.synthetic.main.activity_main.*
```

Manipulation des Views – View Binding

118

- Aujourd'hui Google a encore ajouté la notion de **View Binding** pour remplacer findViewById() aussi bien dans les codes Java que dans les codes Kotlin.
  - Pour activer le View Binding, il suffit d'ajouter cette configuration dans le fichier build.gradle (niveau module) sous la section *android*.

```
android {

    buildFeatures{
        viewBinding = true
    }

    compileSdk 32
```

Manipulation des Views – View Binding

119

- Une fois activé, le View Binding générera automatiquement une classe de liaison pour chacun des fichiers de layout.
  - Nom de la classe de l'activité : **MainActivity**
  - Nom du fichier de layout : **activity_main**
  - Nom de la classe de liaison : **ActivityMainBinding**
    - **Invisible dans l'arborescence du projet.**
- La classe de liaison contient une propriété pour chaque objet d'interface ayant un id.

Manipulation des Views – View Binding

120

- Dans l'activité :
  - On récupère l'interface à partir de la classe de liaison.
  - On la stocke dans un objet de liaison
  - On charge le layout de l'objet de liaison avec setContentView(binding.root).
    - L'objet root correspond généralement au ConstraintLayout principal.

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val binding = ActivityMainBinding.inflate(layoutInflater)

        setContentView(binding.root)

        binding.
```

```
tvHello                                          TextView
btn1                                               Button
edPerson                                         EditText
toString()                                         String
to(that: B) for A in kotlin   Pair<ActivityMainBinding, B>
root (from getRoot())                      ConstraintLayout
```

Manipulation des Views

121

- Tableau comparatif des différentes méthodes

	Elegance	Correct Type of View	Build Speed	Support Kotlin & Java
findViewById	✗	✗	✓	✓
Kotlin Synthetic	✓	✓	✓	✗
ViewBinding	✓	✓	✓	✓

Gestion des évènements

122

- Se fait par l'intermédiaire des listeners.
  - Objets permettant de récupérer l'évènement et le traiter.
  - Interfaces dont chaque méthode correspond à une action qu'il est possible de faire à partir d'un évènement.
  - Par exemple, pour intercepter l'évènement « clic » sur un bouton, on applique l'interface **View.*OnClickListener*** sur ce bouton.
  - Cette interface contient la méthode **fun onClick(v: View?)**
    - Le paramètre de type View est la vue sur laquelle le clic a été effectué.
  - Il existe plusieurs méthodes pour utiliser ces Listeners.

Gestion des évènements – Listener en code Kotlin

123 Méthode 1

- Exemple: Clic sur un bouton
- Capturer tous les clics de bouton dans l'activité
  - La classe de l'activité doit implémenter ***View.OnClickListener***: Méthode par **HÉRITAGE.**

```
class MainActivity : AppCompatActivity(), View.OnClickListener {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val b = findViewById<Button>(R.id.button)
        b.setOnClickListener(this)
    }

    override fun onClick(v: View?) {
        TODO( reason: "Not yet implemented")
    }
}
```

```
override fun onClick(v: View?) {
    TODO( reason: "not implemented")
}
```

Gestion des évènements – Listener en code Kotlin

124 Méthode 1

- Affichage d'un message pop-up.

```
override fun onClick(v: View?) {
    if (v != null) {
        Toast.makeText(v.context, text: "clic sur bouton",Toast.LENGTH_LONG).show()
    }
}
```



Gestion des évènements – Listener en code Kotlin

125 Méthode 1

- Un problème se pose !!!
  - Si on appuie sur un bouton quelconque, on rentre dans la même méthode onClick().
- Intérêt de la vue passée dans la méthode **onClick(v: View?)**
  - Permet de différencier les boutons.

```
override fun onClick(v: View?) {
        when (v?.id) {
            R.id.button -> Toast.makeText(v.context, text: "clic sur bouton 1",Toast.LENGTH_LONG).show()
            R.id.button2 -> Toast.makeText(v.context, text: "clic sur bouton 2",Toast.LENGTH_LONG).show()
        }
}
```

Gestion des évènements – Listener en code Kotlin

126 Méthode 2

- Même exemple autrement

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val b = findViewById<Button>(R.id.button)
        b.setOnClickListener(object :View.OnClickListener{
            override fun onClick(v: View?) {
                Toast.makeText(v?.context,"clic sur bouton",Toast.LENGTH_LONG).show()
            }

        })
    }
}
```

- Méthode par **classe anonyme** ou **object expression** (instancier un objet qui dérive d'une superclasse ou implémente une interface).

Gestion des évènements – Listener en code Kotlin

127 Méthode 2

- Utilisation des lambdas

```
b.setOnClickListener(object :View.OnClickListener{
    override fun onClick(v: View?) {
        Toast.makeText(v?.context,"cl
    }
```

Convert to lambda

Convert to lambda Alt+Maj+Entrée More actions... Alt+Entrée

lambda

```
b.setOnClickListener { v:View? ->
    Toast.makeText(v?.context,"clic sur bouton",Toast.LENGTH_LONG).show() }
```

ou

```
b.setOnClickListener { v ->
    Toast.makeText(v?.context,"clic sur bouton",Toast.LENGTH_LONG).show() }
```

ou

```
b.setOnClickListener { it: View!
    Toast.makeText(this,"clic sur bouton",Toast.LENGTH_LONG).show() }
```

Gestion des évènements – Listener en code XML

128

- Créer le listener dans le code XML

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:onClick="buttonClick"
```

Résultat



- Gérer l'évènement dans le code Kotlin

```
class MainActivity:AppCompatActivity(){

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

    }

    fun buttonClick (view: View?){
        if (view != null) {
            Toast.makeText(view.context, text: "clic sur bouton",Toast.LENGTH_LONG).show()
        }
    }
}
```

Gestion des évènements

129

- Exemple de passage d'une activité à une autre
  - On crée un objet **Intent** (moyen pour faire communiquer des applications, des activités, etc.)

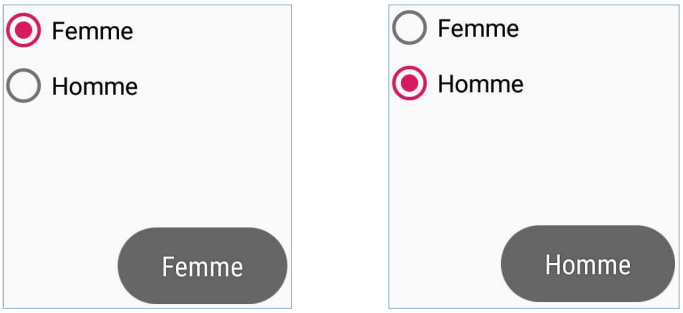
```
fun buttonClick (view: View?){
    val i = Intent(view?.context, Main2Activity::class.java)
    startActivity(i)
}
```

- Le premier argument représente le contexte de l'activité de départ (this / applicationContext / view.context).
- Le second représente l'activité d'arrivée.
- La méthode « **startActivity** » lance l'activité sollicitée via l'intent créé précédemment.

Gestion des évènements

130

- Boutons Radios dans un « Radio Group »
  - Exemple: On cherche à avoir le comportement suivant:



Gestion des évènements

131

- Boutons Radios dans un « Radio Group »
  - Code XML

```
<RadioGroup
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/rg"
    android:layout_marginStart="87dp"
    android:layout_marginTop="72dp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/button2">

    <RadioButton
        android:id="@+id/radioButton1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/femmme"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/button2" />
    <RadioButton
        android:id="@+id/radioButton2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/homme"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/radioButton1" />
</RadioGroup>
```

Gestion des évènements

132

- Boutons Radios dans un « Radio Group »
  - Code Kotlin avec object expression

```
class MainActivity:AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rg = findViewById<RadioGroup>(R.id.rg)
        rg.setOnCheckedChangeListener(object:RadioGroup.OnCheckedChangeListener{
            override fun onCheckedChanged(group: RadioGroup?, checkedId: Int) {
                when (checkedId) {
                    R.id.radioButton1 -> Toast.makeText(applicationContext,  text: "Femme", Toast.LENGTH_LONG).show()
                    R.id.radioButton2 -> Toast.makeText(applicationContext,  text: "Homme", Toast.LENGTH_LONG).show()
                }
            }
        })
    }
}
```

Gestion des évènements

133

- Boutons Radios dans un « Radio Group »
  - Code Kotlin avec lambda

```
class MainActivity:AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rg = findViewById<RadioGroup>(R.id.rg)
        rg.setOnCheckedChangeListener { radioGroup: RadioGroup, i: Int ->
            when (i) {
                R.id.radioButton1 -> Toast.makeText( context: this,  text: "Femme", Toast.LENGTH_LONG).show()
                R.id.radioButton2 -> Toast.makeText( context: this,  text: "Homme", Toast.LENGTH_LONG).show()
            }
        }
    }
}
```

Layouts

134

- Les layouts se trouvent dans le répertoire res/layout
  - Utilisables dans le projet avec la méthode **setContentView()**
    - setContentView(R.layout.activity_main);

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

Layouts

135

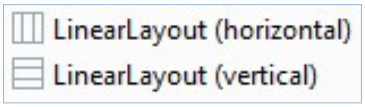
- Leurs définitions se font principalement en XML (interfaces statiques).
  - La définition en Java ou en Kotlin convient aux interfaces dynamiques.
  - La combinaison des deux types est possible.



Layouts – Types

136

- **LinearLayout**: organise les différents éléments de l'interface sur:
  - une ligne (*Horizontal*: de gauche à droite)
  - une colonne (*Vertical*: du haut vers le bas).



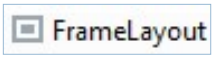
- **RelativeLayout**: permet de placer un élément en fonction du layout parent et par rapport à la position des autres éléments.



Layouts – Types

137

- **FrameLayout**: utilisé pour afficher une seule vue à la fois.
  - Peut contenir de nombreux widgets, mais un seul apparaît.
  - Permet de jouer sur leur visibilité.



Layouts – Types

138

- **ConstraintLayout**: vient d'un nouveau package de widgets

```
import androidx.constraintlayout.widget.ConstraintLayout
```

- Il est comme le RelativeLayout (position par rapport au layout parent ou aux autres éléments), mais c'est plus flexible.
  - Au moins deux contraintes doivent être définies (une verticale et une horizontale) pour chaque vue.
  - Chaque contrainte représente une connexion ou un alignement.
  - Si aucune contrainte n'est définie pour une vue, elle sera affichée à la position (0,0) (coin gauche en haut).



Layouts

139

- Un layout reste toujours modifiable à travers le code.
- Pour cela, si nécessaire, il devient important de spécifier un ***id*** dans la définition XML du gabarit.
  - **android:id**= "**@+id**/idElem"

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/layout"
    android:orientation="horizontal">
</LinearLayout>
```

Configuration – ConstraintLayout

140

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
```

- **android:layout_width** et **android:layout_height**
  - Ces deux propriétés peuvent prendre 3 types de valeur:
    - ***match_parent***: le layout occupe tout l'espace disponible chez son conteneur parent (écran ou autre).
    - ***wrap_content***: le composant prend la taille de son contenu.
    - **Une taille fixe**: par exemple 100dp.

Configuration – ConstraintLayout

141

- **Taille des éléments (Unités) :** valable pour tous les layouts et tous les objets d'interface.
  - dp ou dip (Density-independant Pixel) : il s'agit d'une unité qui est indépendante de la densité de l'écran.
    - Il est déconseillé d'utiliser le pixel (px) ou le millimètre (mm), car ces unités varient d'un écran à l'autre.
    - 200 dp: le widget aura toujours une dimension proportionnelle quelle que soit la taille de l'écran.
    - 200 mm: le widget sera suffisamment grand pour un grand écran et énorme pour un petit écran.
  - sp (Scale-independant Pixel) : unité plus adaptée pour définir la taille d'une police de caractères.
    - Elle respecte le même principe que dp.

Configuration – ConstraintLayout

142

- **ConstraintLayout : Les « guidelines »**
  - Une Guideline est un composant virtuel (qui n'apparait pas sur l'écran) représentant une ligne de référence, verticale ou horizontale, sur laquelle les vues pourront venir s'ancrer.
  - Une *Guideline* peut être placée de 3 façons :
    - Distance en « *dp*» depuis le haut (pour une orientation horizontale) ou du côté gauche (pour une orientation verticale)

```
app:layout_constraintGuide_begin="100dp"
```

    - Distance en « *dp*» depuis le bas (orientation horizontale) ou du côté droit (orientation verticale)

```
app:layout_constraintGuide_end="200dp"
```

    - En pourcentage sur l'axe vertical ou horizontal

```
app:layout_constraintGuide_percent="0.25"
```

Configuration – ConstraintLayout

143

- **ConstraintLayout : Les « groups »**
  - La contrainte « *Groups* » permet de rassembler, de manière transparente sur l'interface, des *vues* sous un identifiant commun.
  - Ceci servira à appliquer la même chose à un ensemble d'éléments graphiques en même temps.
    - La visibilité, une animation particulière, etc.

```
<androidx.constraintlayout.widget.Group
    android:id="@+id/grp"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:constraint_referenced_ids="tvValue2, editValue2"/>
```

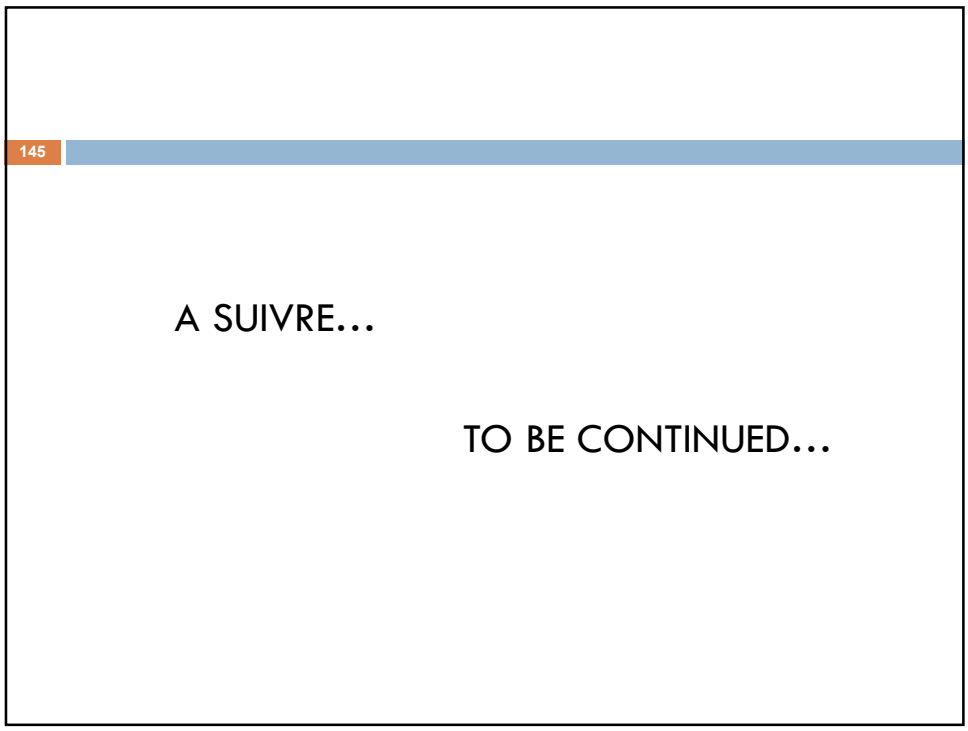
```
grp.visibility=View.INVISIBLE
```

Ceci rend invisibles à la fois les vues ayant les ids « tvValue2 » et « editValue2 »

Pour Finir…

144

- L'IHM est d'une grande importance pour les applications Android.
  - Il faut bien choisir les vues, leurs dispositions, leurs propriétés, etc.
  - Sur un écran à taille limitée, organiser l'IHM n'est pas forcément une chose facile.
    - Trouver des moyens pour la bonne organisation de l'IHM (fragments, onglets, etc.)
- Toute activité suit un cycle de vie et il faut bien gérer les différentes phases.



Naoufel KHAYATI